

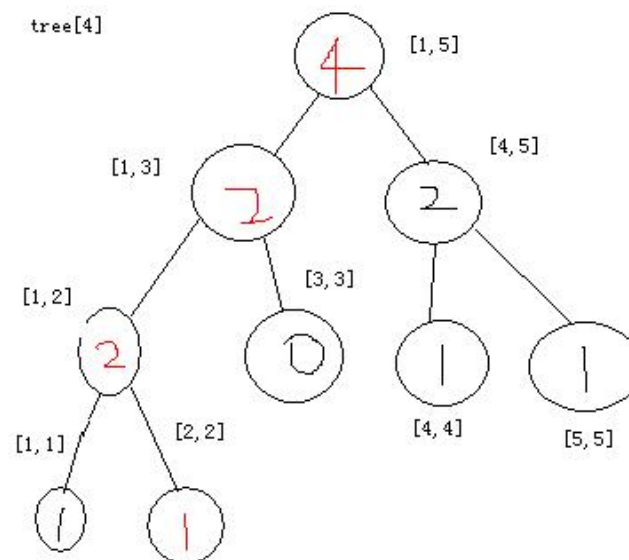
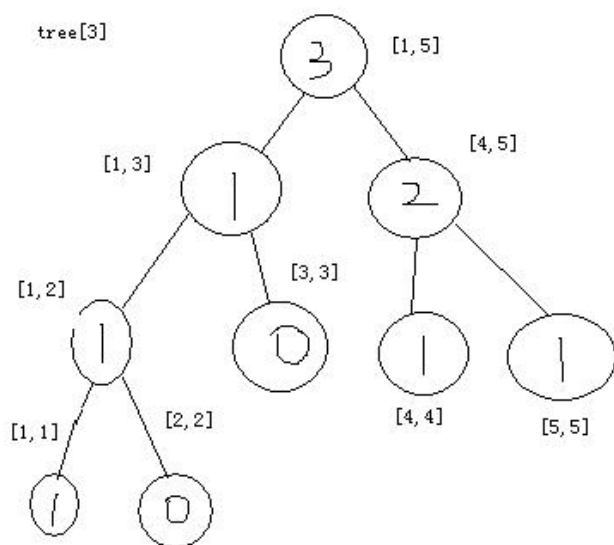
可持久化线段树

函数式线段树

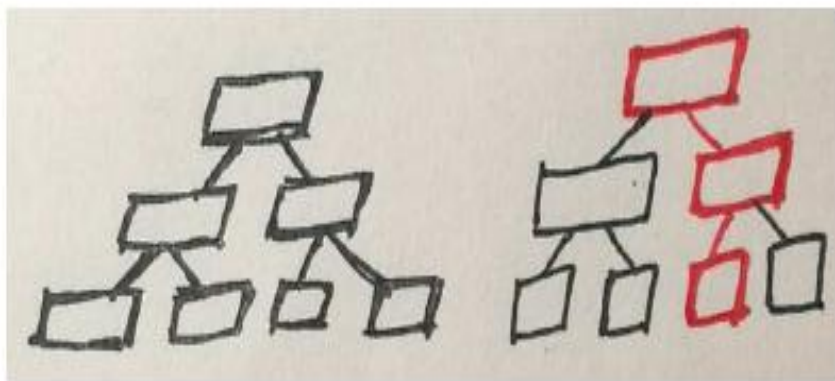
- 什么叫函数式线段树？
- 普通的线段树在更新节点时，直接把原来的信息覆盖
- 而函数式线段树不修改原有信息。更新时影响到的点，全部新开一个版本

函数式线段树

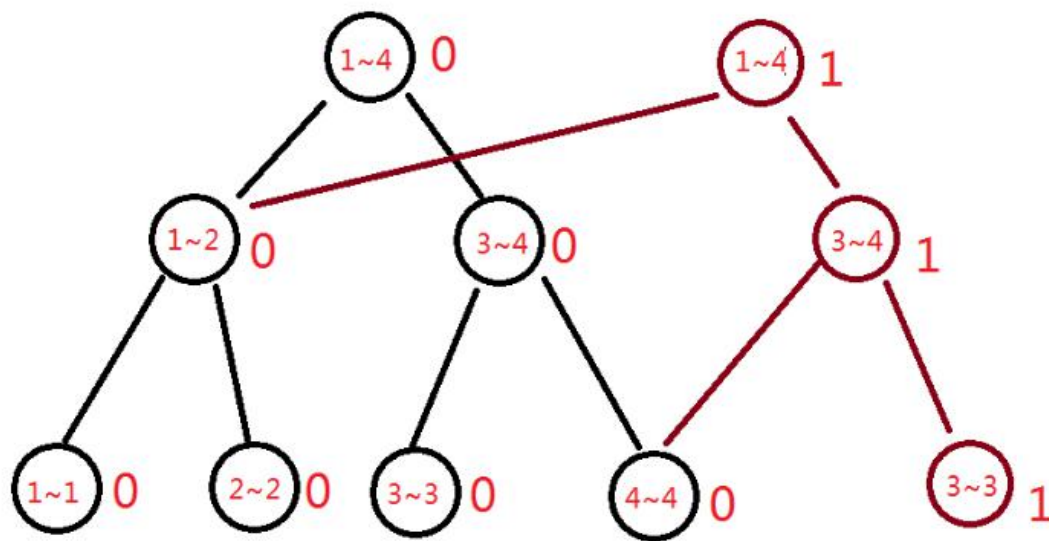
- 什么叫函数式线段树？
- 普通的线段树在更新节点时，直接把原来的信息覆盖
- 而函数式线段树不修改原有信息。更新时影响到的点，全部新开一个版本
- 单点修改时，信息有更新的点：



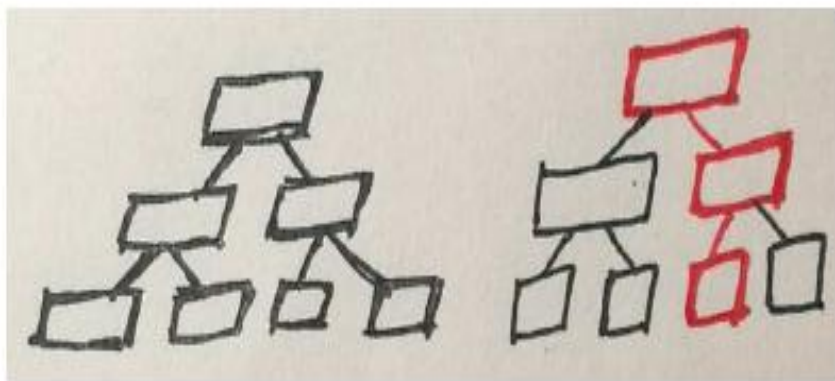
- 新版本具体而言，大概是以下这样：



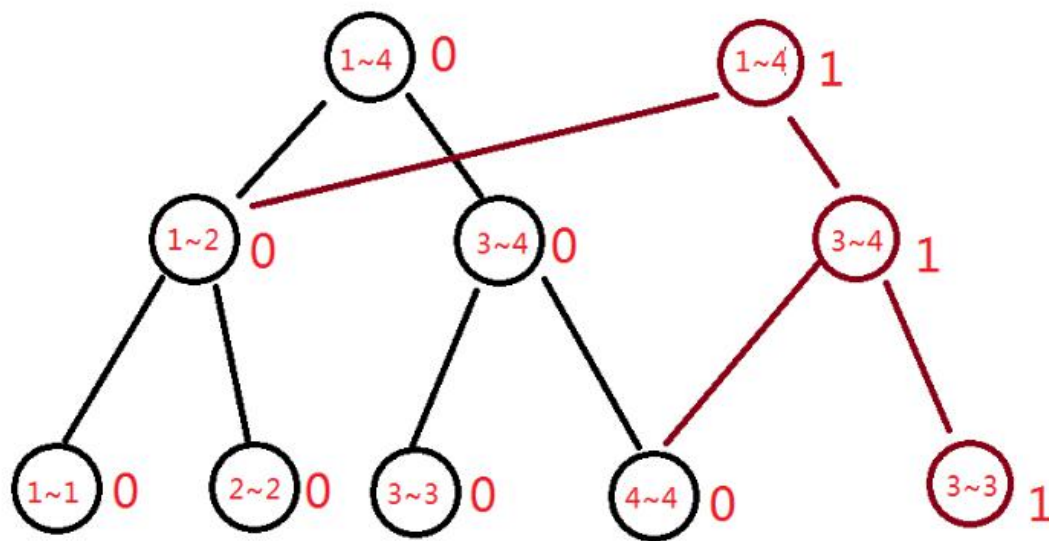
拆开来看：



- 新版本具体而言，大概是以下这样：

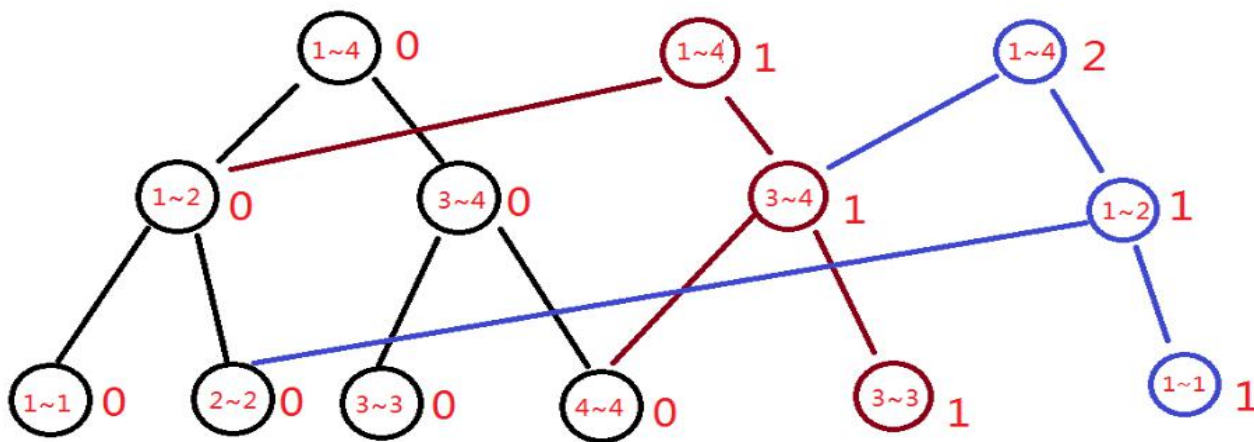


拆开来看：

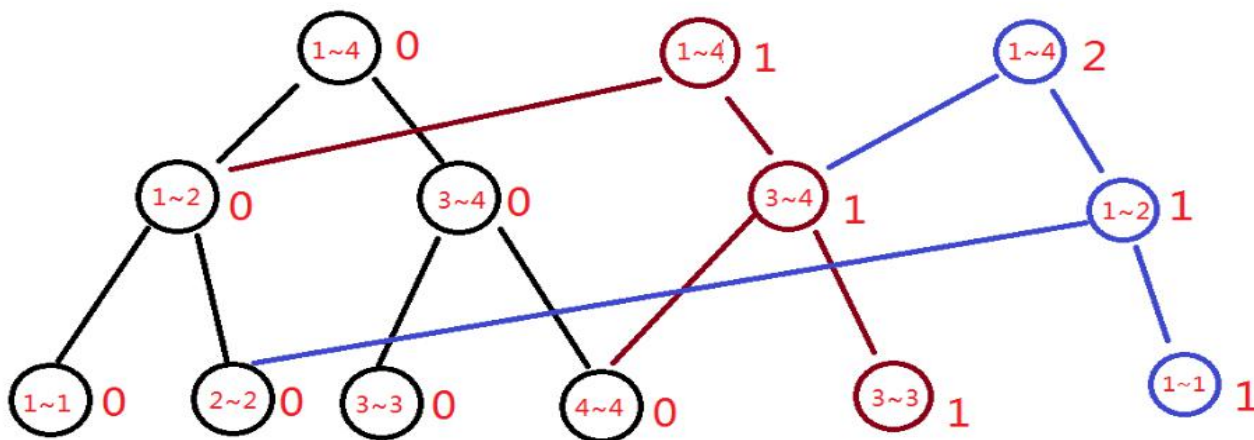


比如根节点的左儿子信息没有变，新版本的左儿子就直接接到旧版本那里就行了

- 新版本并不会开出一整棵线段树，而是其中的一部分。没变的地方直接指向旧版本。有修改才有新开的点。
- 修改多次后：

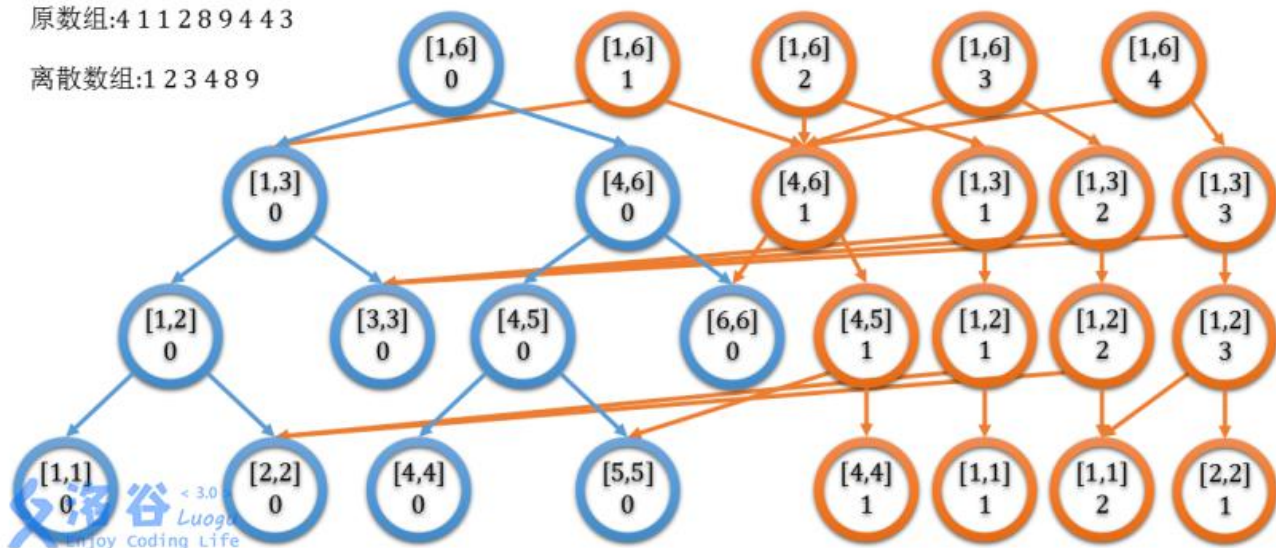


- 新版本并不会开出一整棵线段树，而是其中的一部分。没变的地方直接指向旧版本。有修改才有新开的点。
- 修改多几次后：

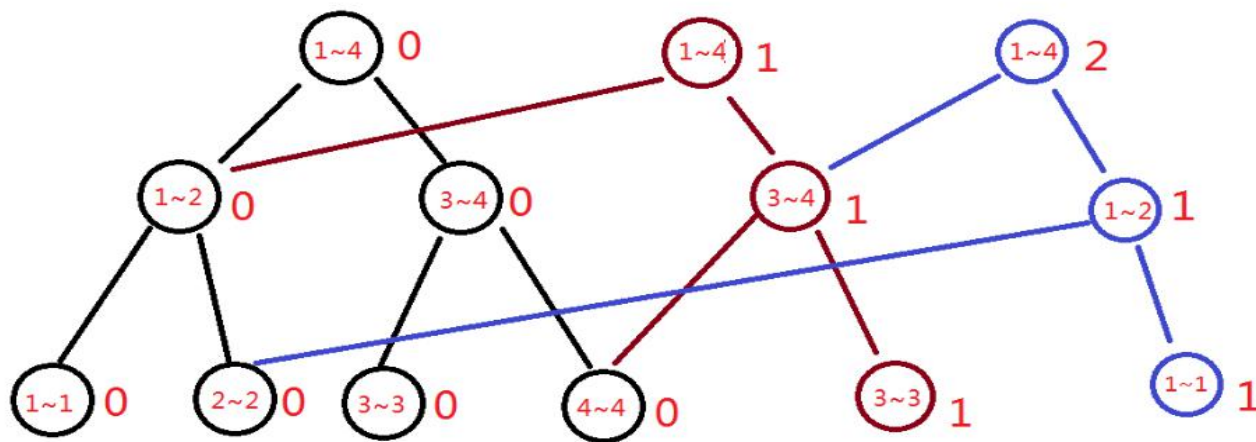


原数组:4 1 1 2 8 9 4 4 3

离散数组:1 2 3 4 8 9

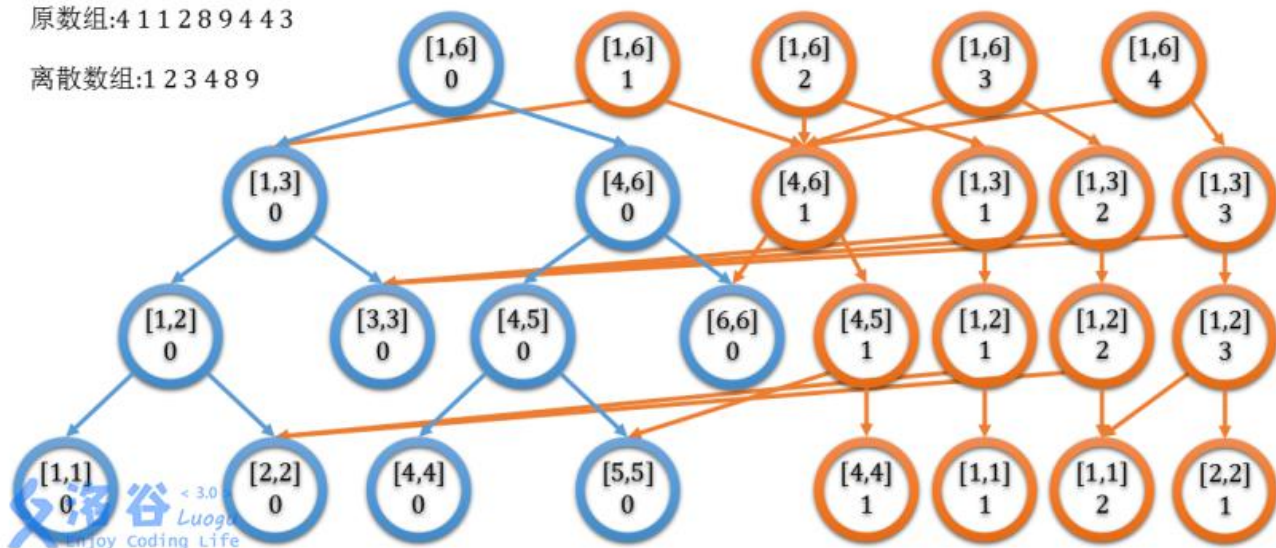


- 新版本并不会开出一整棵线段树，而是其中的一部分。没变的地方直接指向旧版本。有修改才有新开的点。
- 修改多几次后：



原数组:4 1 1 2 8 9 4 4 3

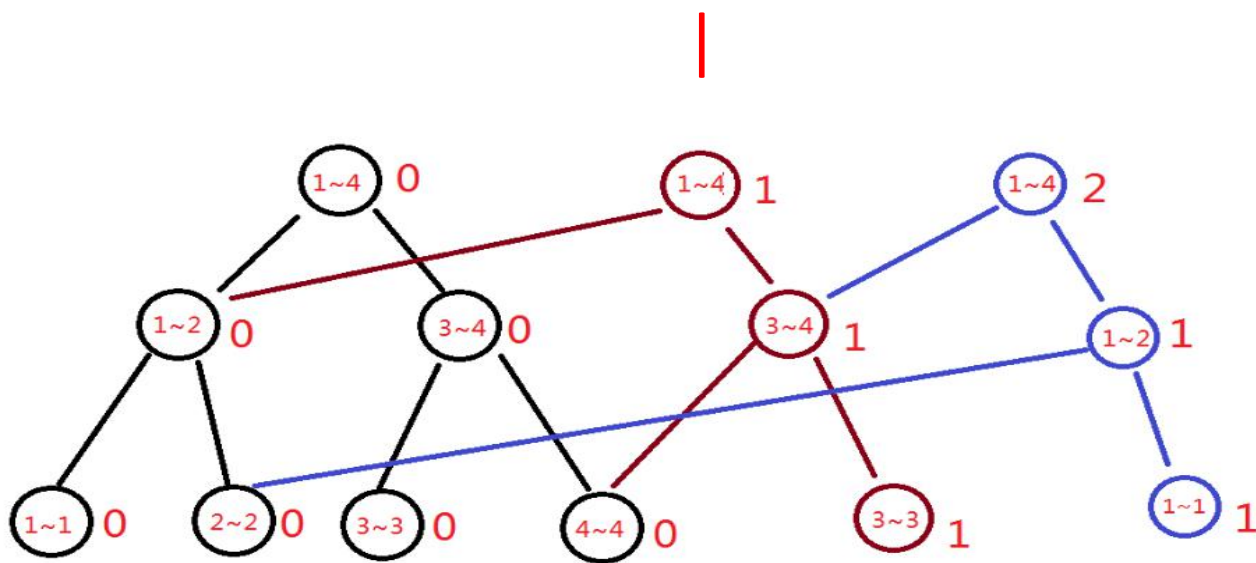
离散数组:1 2 3 4 8 9



密恐请远离

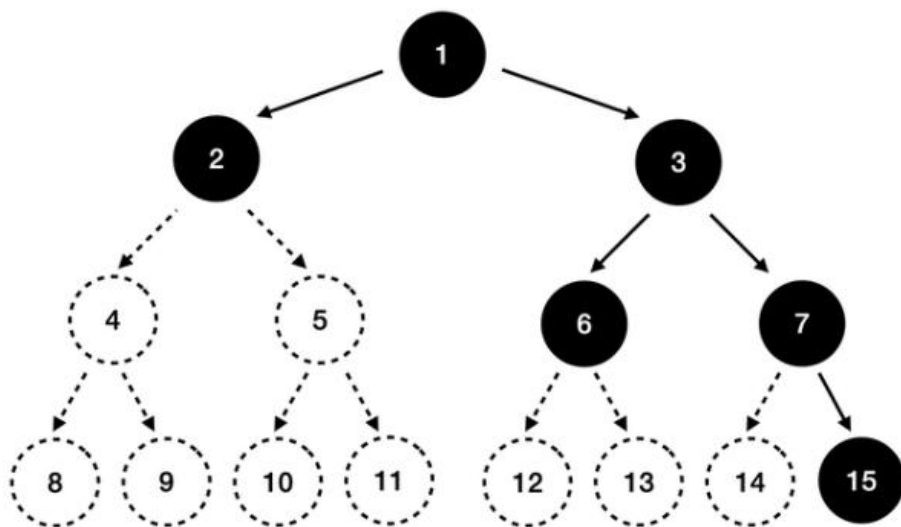
为什么要写函数式线段树？

- 缺点很显然：空间开销很大
- 优点：保留了所有历史版本
- 比如更新了两次之后，忽然又要查询第一次更新后的信息，也能hold得住，直接找对应版本的根节点，往下查就行。



一个要注意的地方

- 先前接触的线段树：节点node的左右儿子，编号必须是 $node*2$ 和 $node*2+1$ 。
- 节点只存储信息，访问左右儿子直接通过下标运算进行。



```
struct Tnode
{
    //信息1
    //信息2
    //... ..
}
```

- 而函数式线段树：先开一个池子，每次要新建节点的时候才从里面拿。每个点要记住自己左右孩子的编号。

```
struct Tnode
{
    Tnode *lson,*rson;
    //信息1
    //信息2
    //... ..
} tree[max_number_of_node];

Tnode *Root[max_m]; //存各个版本线段树的根
```

- 有一个很重要的性质：
- 一个点只会有唯一的左右孩子，但它可能有很多个父亲

现在来计算内存开销

现在来计算内存开销

- 聪明的你一定已经知道，一次单点修改需要开的节点数是 $\log(n)$

现在来计算内存开销

- 聪明的你一定已经知道，一次单点修改需要开的节点数是 $\log(n)$
- 区间修改单点查询，可以通过头+1尾后-1的方法，转化为两次单点修改+区间查询

现在来计算内存开销

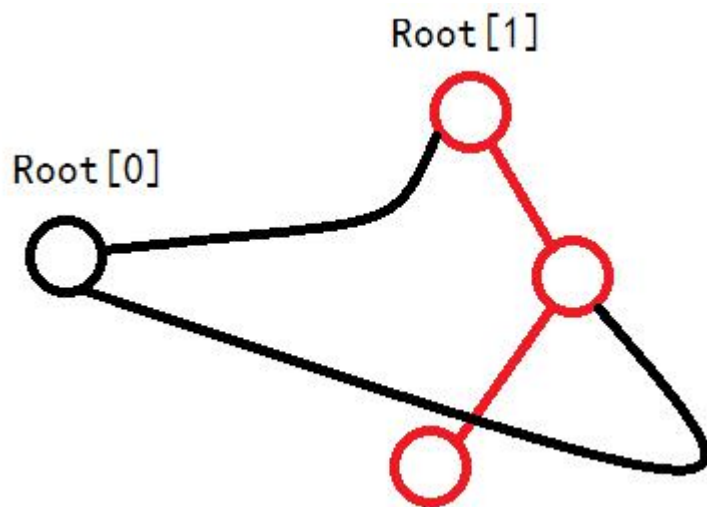
- 聪明的你一定已经知道，一次单点修改需要开的节点数是 $\log(n)$
- 区间修改单点查询，可以通过头+1尾后-1的方法，转化为两次单点修改+区间查询
- 区间修改区间查询，涉及到`lazy_tag`，最坏6个 $\log$ ，建议直接看着内存来开。（一般不这样考）

现在来计算内存开销

- 聪明的你一定已经知道，一次单点修改需要开的节点数是 $\log(n)$
- 区间修改单点查询，可以通过头+1尾后-1的方法，转化为两次单点修改+区间查询
- 区间修改区间查询，涉及到`lazy_tag`，最坏6个 $\log$ ，建议直接看着内存来开。（一般不这样考）
- 空间： $4n+m\log(n)$

具体代码实现—建树

- 有一种trick的方法，以区间求和为例：
- 最开始的版本里，只建一个点，和为0，左右儿子指向自身，就行了
- 好处：完全不用建树，还剩了 $4n$ 的空间
- 但如果不只是求和，要考虑一下这样做有没有bug



查询的时候，查到这个点也会一直在这里打转，所以没有影响

具体代码实现—更新

```
Tnode *Update(Tnode *past,int L,int R,int x,int v)
{
    Tnode *node=New_node(); //创建新节点
    (*node)=(*past); //从旧版本节点复制信息
    if (L==R) node->val=v;
    else
        if (x<=mid) node->lson=Update(past->lson,L,mid,x,v); //递归更新
        else node->rson=Update(past->rson,mid+1,R,x,v);
    return node; //返回新建节点编号
}
```

具体代码实现—查询

```
int Query(Tnode *node,int L,int R,int x)
{
    if (L==R) return node->val;
    if (x<=mid) return Query(node->lson,L,mid,x);
    else return Query(node->rson,mid+1,R,x);
}
```

具体代码实现—查询

```
int Query(Tnode *node,int L,int R,int x)
{
    if (L==R) return node->val;
    if (x<=mid) return Query(node->lson,L,mid,x);
    else return Query(node->rson,mid+1,R,x);
}
```

就直接查

模板题：洛谷P3919

经典的静态区间第k大问题

- 给出长度为 n 的非负序列， m 次求区间 $[l,r]$ 中，第 k 大的数是多少。
- $n, m \leq 2 * 10^5$

经典的静态区间第k大问题

- 给出长度为 n 的非负序列， m 次求区间 $[l,r]$ 中，第 k 大的数是多少。
- $n, m \leq 2 * 10^5$
- Tips1: 从二分答案开始

经典的静态区间第k大问题

- 给出长度为 n 的非负序列， m 次求区间 $[l,r]$ 中，第 k 大的数是多少。
- $n,m \leq 2 \times 10^5$
- Tips1: 从二分答案开始
- Tips2: 建出 $a[1], a[2], \dots, a[i]$ 的权值线段树

经典的静态区间第k大问题

- 给出长度为 n 的非负序列， m 次求区间 $[l,r]$ 中，第 k 大的数是多少。
- $n,m \leq 2 \times 10^5$
- Tips1: 从二分答案开始
- Tips2: 建出 $a[1], a[2], \dots, a[i]$ 的权值线段树
- 洛谷P3834

树上路径第k大

- 给出一颗有 n ($n \leq 10^5$) 个节点的树，每一个节点有一个权值 $v[i]$ ($v[i] \leq 10^9$)。
- m ($m \leq 10^5$) 个询问，每一次问 $u \rightarrow v$ 的路径中第 k 小的点权，保证 k 合法。
- 强制在线

树上路径第k大

- 给出一颗有 n ($n \leq 10^5$) 个节点的树，每一个节点有一个权值 $v[i]$ ($v[i] \leq 10^9$)。
- m ($m \leq 10^5$) 个询问，每一次问 $u \rightarrow v$ 的路径中第 k 小的点权，保证 k 合法。
- 强制在线
- 思考：如何通过加减法得到路径上所有点值组成的线段树

树上路径第k大

- 给出一颗有 n ($n \leq 10^5$) 个节点的树，每一个节点有一个权值 $v[i]$ ($v[i] \leq 10^9$)。
- m ($m \leq 10^5$) 个询问，每一次问 $u \rightarrow v$ 的路径中第 k 小的点权，保证 k 合法。
- 强制在线
- 思考：如何通过加减法得到路径上所有点值组成的线段树
- bzoj2588

扩展题

- Bzoj2809/洛谷P1552
- Hdu5919
- Bzoj4408/洛谷P4587

扩展题

- Bzoj2809/洛谷P1552
- Hdu5919
- Bzoj4408/洛谷P4587
- 参考代码和题解见目录下